Chichi

Technical Documentation

Random video chat platform with integrated board games & social games

Built on Django · Django Channels · WebRTC · Redis

Contents

01 — Architecture Overview

What Is Chichi?

Chichi is a random video chat platform (think Omegle / OmeTV / Monkey) built on Django + Django Channels. Users are matched with strangers for live video calls. On top of the video layer, they can play board games together in real time. Social/party games are planned for a future phase.

High-Level Stack

The diagram below shows every layer of the system and how they connect:

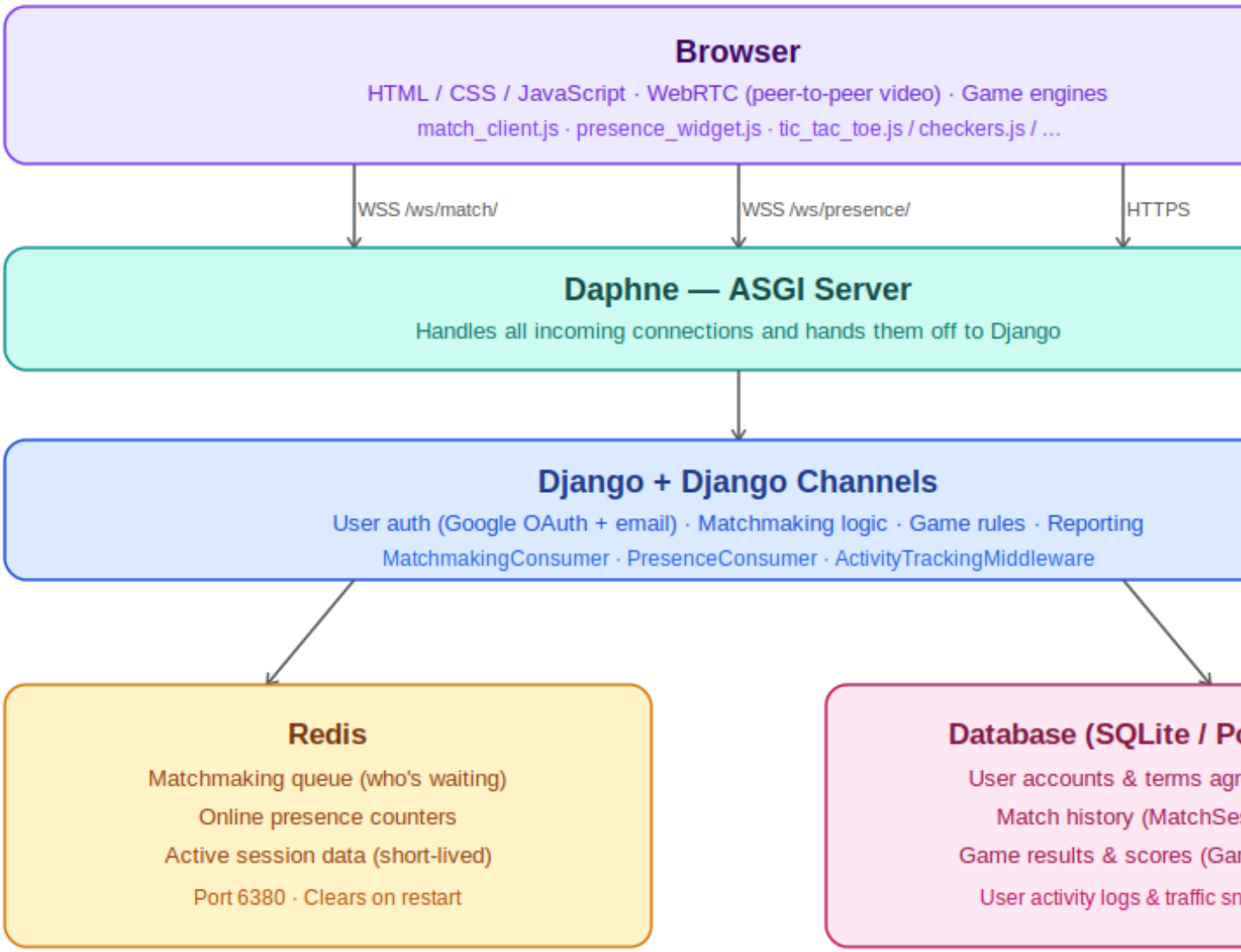

Figure 1 — Chichi system architecture

Key Technology Choices

Layer	Technology	Reason
Web framework	Django 6.0	Batteries-included, ORM, admin
Async server	Daphne (ASGI)	Required for Django Channels
WebSocket	Django Channels	Native Django WS + channel layer
Real-time broker	Redis	Channel layer backend + matchmaking pool
Auth	allauth + CustomUser	Google OAuth + email/password
Video	WebRTC (browser-native)	Peer-to-peer; no media server needed
Deployment	Nginx → Daphne, systemd	Standard production setup

Two WebSocket Endpoints

/ws/match/ — MatchmakingConsumer

The "brain" of the platform. Handles:

- Matchmaking via Redis pools (vibe-scoped)
- WebRTC signalling (offer / answer / ICE candidates)
- In-session messaging (chat, game moves, sync)
- Game lifecycle (start / over / quit / forfeit)
- Moderation (report_peer)
- Presence side-effect (online count)

/ws/presence/ — PresenceConsumer

Read-only fan-out. Any page can subscribe to get live online_count and online_users_count pushed down whenever any user connects or disconnects.

Vibe System

Every matchmaking session starts with a vibe selection. The user declares what kind of session they want before being matched:

Vibe	Pool Key (Redis)	Description
casual	matchmaking:pool:casual	Random video chat, no game
board_game	matchmaking:pool:board_game:<code>	Video + auto-launched game

Supported game codes:

- ttt — Tic Tac Toe
- 3mm — Three Men's Morris
- cks — Checkers
- sdk — Sudoku

> *Adding a new game = add a row to GameType in the DB + add a JS engine file + add the code to GAME_LABELS in consumers.py.*

02 — Event Flow

All real-time events travel over the /ws/match/ WebSocket between the browser and Django Channels (MatchmakingConsumer). Within the server, events travel through the Redis channel layer between two consumer instances.

Matchmaking Flow

The diagram below shows how two users find each other, negotiate a video connection, and optionally start a game:

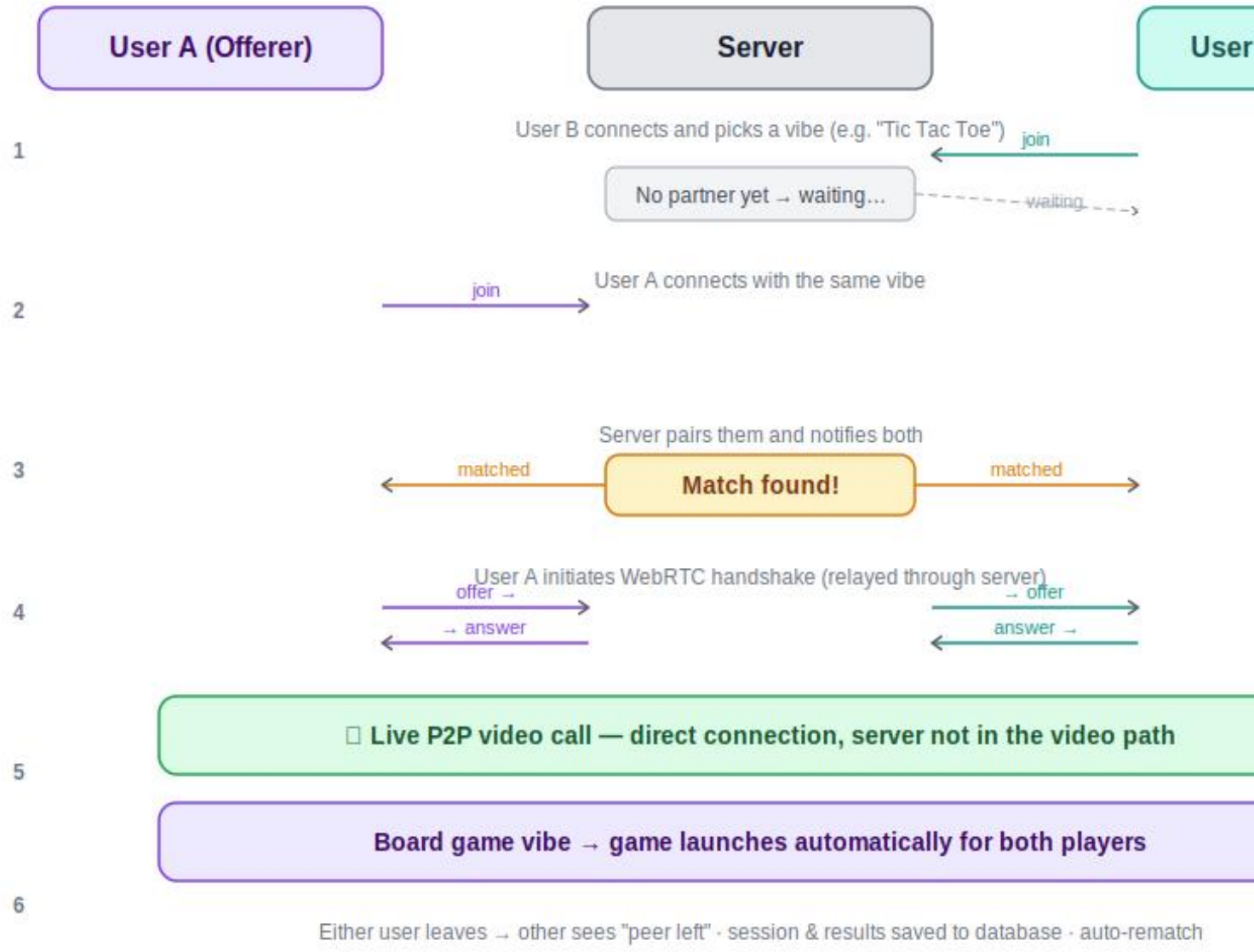

Figure 2 — Matchmaking and WebRTC connection flow

Key Events Reference

Direction	Event	Payload
Client → Server	join	{ type, vibe, game? }
Server → Client	waiting	{ type, vibe, game, game_label }
Server → Offerer	matched	{ session_id, is_offerer:true, vibe, game, game_label }
Server → Answerer	matched	{ session_id, is_offerer:false, vibe, game, game_label }
Server → Both	game_start	{ type, game_type, auto:true } (board_game only)
Client → Server	offer / answer / ice_candidate	WebRTC signalling, relayed verbatim
Server → Client	peer_left	{ type } — sent when partner disconnects

In-Session Messaging

Once matched, either peer can send these events — they are forwarded to the other peer via group_send:

Event type	Channel layer type	Purpose
chat	session.chat	Text chat message
game_move	session.game_move	Board game move
sync	session.sync	Game state sync (board reconciliation)
custom	session.custom	Extensible; future use (reactions, etc.)

Disconnect & Cleanup

When a user disconnects:

- Presence counter is decremented in Redis
- If waiting in pool: removed from pool
- If in-game: forfeit recorded in DB
- If in-session: MatchSession end_time and duration saved; peer_left sent to partner
- Redis session key deleted; channel group discarded

03 — Sequence Diagrams

The diagrams in this section use Mermaid syntax and can be rendered in GitHub, Notion, or any Mermaid-compatible viewer.

SD-01: Full Session Lifecycle (Casual Video Chat)

> *User B connects first and is placed in the waiting pool. User A connects and triggers the match. A becomes the offerer (initiates WebRTC). After signalling, video is P2P. When A disconnects, the session is closed in the DB and B receives peer_left.*

Key sequence steps:

- B joins → server finds empty pool → B waits
- A joins → server finds B → creates MatchSession in DB → both get matched
- A creates WebRTC offer → server relays to B → B answers → ICE exchange
- P2P video active (server no longer in video path)
- A disconnects → session closed → B receives peer_left

SD-02: Board Game Session (Auto-Start)

> *When both users select the same board_game vibe and game code, the server auto-sends game_start to both after matching. The game launches immediately without any user interaction.*

Additional steps after matching:

- Server sends game_start { game_type, auto:true } to both peers
- Both game engines initialise and render the board
- Moves exchanged as game_move events relayed through server
- Either peer sends game_over → offerer resolves → DB write → game_result to both

SD-03: Peer Report

- Reporter sends { type:"report_peer" }
- Server sets MatchSession.reported = True, reported_by, reported_at
- Server replies with report_ack to reporter only

> *No notification is sent to the reported peer.*

SD-04: ICE Watchdog Timeout & Rematch

- Client-side watchdog fires after 5 seconds if ICE connection fails
- Client sends watchdog_timeout → server closes session, sends peer_left
- Server replies with watchdog_ack
- Client auto-rematches after 1 second delay

SD-05: Game Forfeit on Disconnect

- Player closes tab or connection drops
- Server records GameResult with loser_forfeit=True for both players
- MatchSession end_time saved
- Remaining partner receives peer_left → auto-rematch triggered

04 — State Transitions

Connection States (Server-Side)

Each WebSocket connection moves through these states from connect to disconnect:

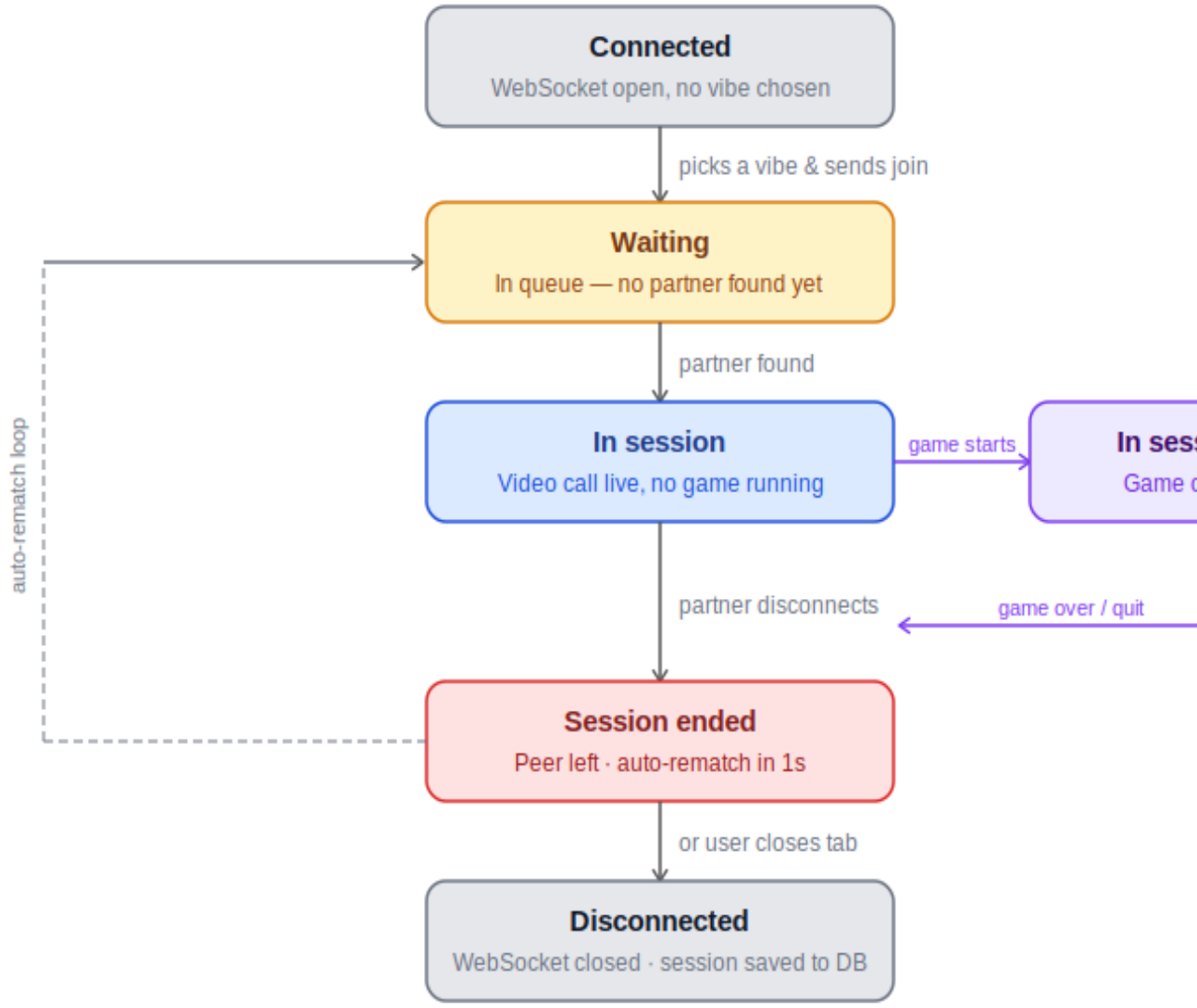

Figure 3 — Connection state machine

State Variable Summary

State	session_id	session_group	game_session_id	game_type	vibe
Connected	None	None	None	None	None
Waiting	None	None	None	None	set
In session (no game)	set	set	None	None	set

State	session_id	session_group	game_session_id	game_type	vibe
In session + game	set	set	set	set	set
Disconnected	cleared	cleared	cleared	cleared	any

Game Lifecycle States

When a game is played:

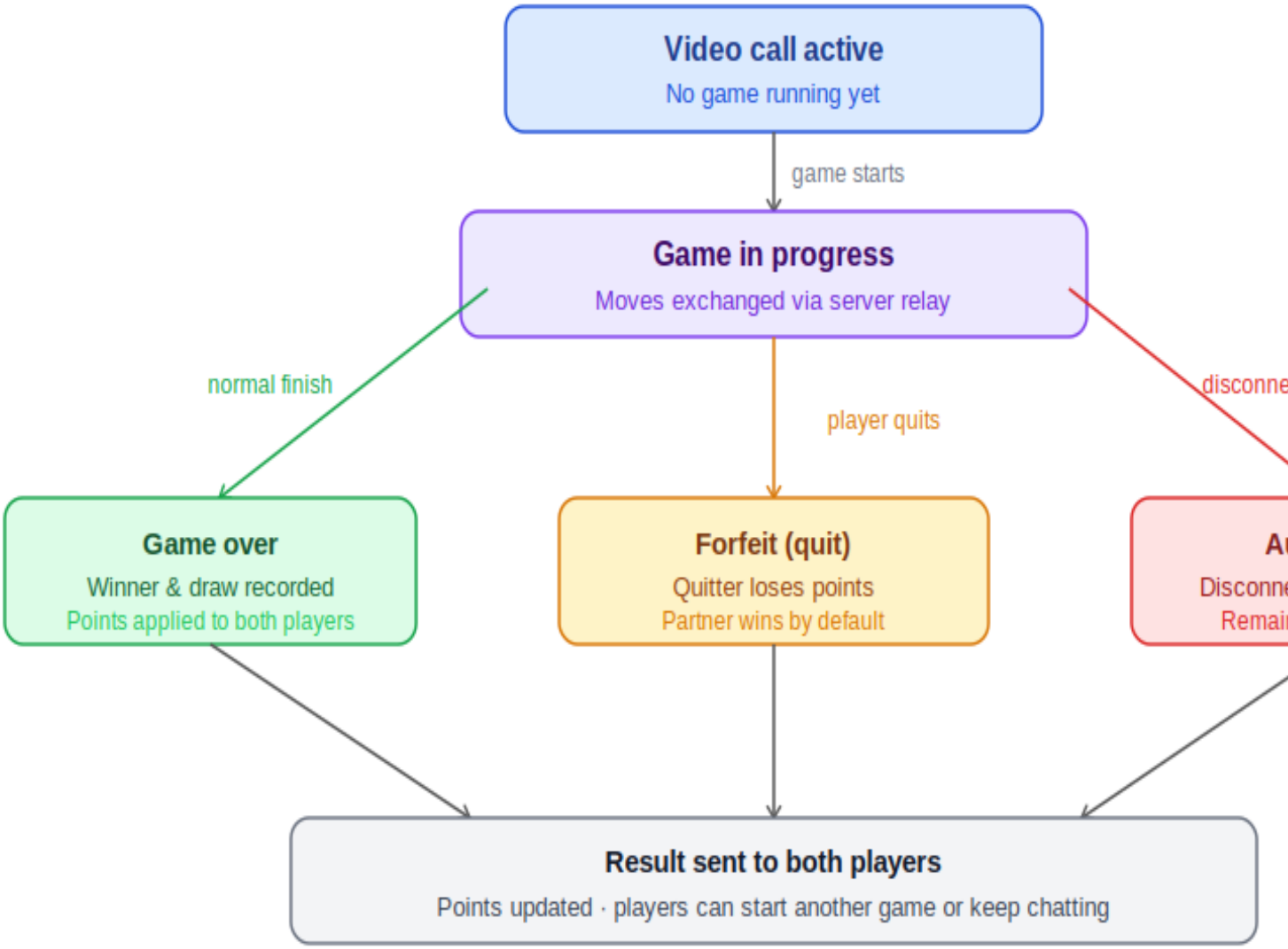

Figure 4 — Game lifecycle and outcomes

Points System

Win	+3 points (configurable per game type)

Draw	+1 point
Loss	-1 point
Forfeit (loser)	-1 point
Forfeit (winner)	+3 points

05 — Data Models

Entity Overview

CustomUser is the central entity. All other models link back to it:

- CustomUser → UserTermsAndPolicy (1:1): terms acceptance gate
- CustomUser → MatchSession (FK x2): user_one and user_two per session
- MatchSession → GameResult (1:N): two rows per game (one per player)
- GameResult → GameType (FK): which game was played

authentication.CustomUser

Extends Django's AbstractUser. Email is the USERNAME_FIELD.

username	CharField(100) — display name
email	EmailField(100) — unique, used for login
password	Explicit CharField(100) on the model — see Technical Debt #6

authentication.UserTermsAndPolicy

One-to-one with CustomUser. Must exist and be accepted before accessing /connect/.

terms_and_conditions_agreed	BooleanField — must be True to connect
age_confirmation	BooleanField — must be True to connect (18+)
promotional_emails_agreed	BooleanField — optional marketing consent

core.MatchSession

One row per matched pair. Written on match, closed on disconnect.

session_id	UUIDField (unique) — mirrors Redis session key
user_one	FK → CustomUser — the answerer (already in pool)
user_two	FK → CustomUser — the offerer (triggered the match)
start_time / end_time	DateTimeField — auto-set; end_time set on first disconnect
duration_seconds	PositiveIntegerField (nullable)
reported / reported_by / reported_at	Set by report_peer event; filterable in Django admin

core.GameType

Plugin descriptor for each mini-game. Add a row + a JS file to ship a new game.

name	CharField(32, unique) — short code, e.g. "ttt"
display_name	CharField(64) — human label, e.g. "Tic Tac Toe"

win_points / draw_points / loss_points	IntegerField — defaults: 3 / 1 / -1
active	BooleanField — inactive types are not offered

core.GameResult

Two rows written per game (one per player). Immutable after creation.

session	FK → MatchSession
game_type	FK → GameType
user / opponent	FK → CustomUser
is_win / is_draw / is_loss / is_forfeit	BooleanField
points_earned	IntegerField — positive or negative delta
timestamp	DateTimeField — auto-set

Redis Key Reference

Key	Type	Purpose
matchmaking:pool:casual	Set	Waiting users for casual vibe
matchmaking:pool:board_game:<code>	Set	Waiting users for a specific game
session:<uuid>	Hash	Live session metadata; TTL 1 hour
presence:online_count	String (int)	Total open WS connections
presence:online_users	Set	Authenticated user PKs
presence:user_conns:<pk>	String (int)	Per-user tab/device ref count

06 — WebSocket Protocol

All messages are JSON over ws[s]://<host>/ws/match/. Direction: C→S = Client to Server | S→C = Server to Client.

Connection Lifecycle

join — C→S

First message after WebSocket opens. Declares vibe and optional game.

```
{ "type": "join", "vibe": "casual", "game": null }
{ "type": "join", "vibe": "board_game", "game": "ttt" }
```

Valid vibe values: "casual", "board_game". Valid game codes: "ttt", "3mm", "cks", "sdk"

waiting — S→C

Sent when no partner is immediately available.

```
{ "type": "waiting", "vibe": "board_game", "game": "ttt", "game_label": "Tic Tac Toe"
}
```

matched — S→C

Sent to both peers when a match is found. is_offerer:true means this client initiates the WebRTC offer.

```
{ "type": "matched", "session_id": "550e...", "is_offerer": true, "vibe":
"board_game", "game": "ttt", "game_label": "Tic Tac Toe" }
```

peer_left — S→C

Sent to the remaining peer when the other disconnects. Optionally includes "reason": "watchdog" for ICE timeout cases.

WebRTC Signalling

offer, answer, and ice_candidate messages are all forwarded verbatim to the other peer. The server does not interpret SDP.

Game Events

Event	Direction	Description
game_start	C→S or S→C	Manual (user-initiated) or auto (board_game vibe). auto:true skips confirmation dialog.
game_move	C→S (relayed)	Game-engine-specific payload. Server treats as opaque and relays to partner.
sync	C→S (relayed)	Full board state sync. Used for reconciliation.

Event	Direction	Description
game_over	C→S	Either peer sends. winner: "offerer" | "answerer" | "draw". Only offerer resolves server-side.
game_result	S→C	Sent to both peers after resolution. Includes winner, is_draw, game_type, session_id. Forfeits include forfeit:true.
game_quit	C→S	Player voluntarily quits mid-game (forfeit).

Other Events

Event	Direction	Description
report_peer	C→S	Flags the session as reported. No notification to reported peer.
report_ack	S→C	Confirms report received. Includes session_id.
chat	C→S (relayed)	Text chat message to partner.
watchdog_timeout	C→S	Sent by client after 5s of failed ICE connectivity.
watchdog_ack	S→C	Confirms watchdog received; client auto-rematches after 1s.
error	S→C	General error. Includes message field, e.g. "Invalid vibe: party".
custom	C→S (relayed)	Reserved for future use (reactions, etc.). Pass any JSON payload.

Presence WebSocket (/ws/presence/)

Read-only. No messages sent from client. Server pushes on connect and on every user connect/disconnect:

```
{ "type": "presence", "online_count": 42, "online_users_count": 38 }
```

07 — Game Engine Guide

Architecture

Games in Chichi sit on top of the video session. The WebRTC P2P connection is always established first; games then run as a UI overlay, with moves exchanged as game_move messages relayed through the server.

The server treats all move payloads as opaque and simply relays them to the other peer. All game logic lives in the browser JS engines.

Currently Shipped Games

Code	Name	File
ttt	Tic Tac Toe	games/tic_tac_toe.js
3mm	Three Men's Morris	games/three_mens_morris.js
cks	Checkers	games/checkers.js
sdk	Sudoku	games/sudoku.js

Adding a New Game — Checklist

Step 1: Add a GameType row via Django admin or a data migration.

```
GameType.objects.create(name="chess", display_name="Chess", win_points=3,
draw_points=1, loss_points=-1, active=True)
```

Step 2: Register the code in consumers.py.

```
GAME_LABELS = { ..., "chess": "Chess" }
```

Step 3: Create the JS engine at core/static/files/games/chess.js.

The engine must expose:

- init(boardId, isOfferer, onGameOver) — initialises and renders the board
- handleRemoteMove(msg) — applies a partner move
- destroy() — cleanup (no separate reset(); engines also return only {init, handleRemoteMove, destroy})
- Call MatchClient.sendGameMove(payload) for local moves
- Call MatchClient.sendGameOver(winner) when the game concludes

Step 4: Wire up in connect.html inside the onGameStart and onGameMove callbacks.

Step 5: Add the game option to the vibe selector UI in connect.html.

match_client.js API for Games

MatchClient.sendGameMove(payload)	Send a move to your partner
MatchClient.sendSync(state)	Send a full board state sync
MatchClient.sendGameOver(winner)	Signal game ended: "offerer" | "answerer" | "draw"
MatchClient.sendGameQuit()	Signal forfeit

Game Roles

When matched on a board game, the server assigns:

- Offerer (is_offerer: true) → typically Player 1 / first mover
- Answerer (is_offerer: false) → typically Player 2

> *This ensures deterministic role assignment with no client-side negotiation needed.*

08 — Deployment & Infrastructure

Server Stack

Internet → Nginx (TLS termination, static files) → Daphne ASGI → Django/Channels → Redis + Database

Key Files

File	Purpose
chichi/asgi.py	ASGI entry point; wires HTTP and WS routing
daphne/daphne.service	systemd service unit for Daphne
daphne/daphne.socket	systemd socket activation
nginx/nginx.conf	Nginx reverse proxy config
scripts/start_app.sh	Starts Daphne (used by deploy)
scripts/after_install.sh	Post-deploy hook
deploy.sh	Full deploy script

Startup Commands (Development)

```
redis-server --port 6380
daphne -b 0.0.0.0 -p 8000 chichi.asgi:application
python manage.py runserver   # HTTP only, no WS — use Daphne for WS
```

Environment Variables

Variable	Default	Notes
REDIS_URL	redis://127.0.0.1:6380/0	Channel layer + matchmaking
SECRET_KEY	—	Django secret key
DEBUG	—	Set False in production
ALLOWED_HOSTS	—	Comma-separated list
DATABASE_URL	—	SQLite or Postgres
GOOGLE_CLIENT_ID	—	Google OAuth client ID
GOOGLE_CLIENT_SECRET	—	Google OAuth secret
EMAIL_*	—	SMTP settings for allauth

WebRTC ICE Servers

Currently configured with public Google STUN servers. For production, add a TURN server (e.g., coturn) for users behind symmetric NAT. Without TURN, ~15–20% of users may fail to establish P2P connections.

Scaling Considerations

Concern	Current	Recommended at scale
WebSocket server	Single Daphne	Multiple Daphne workers behind load balancer
Channel layer	Redis (single)	Redis Cluster or sentinel
Media relay	None (P2P)	TURN server (coturn)
Database	SQLite	PostgreSQL with PgBouncer
Static files	Nginx	CDN (CloudFront / Cloudflare)
Task queue	cron	Celery + Redis/RabbitMQ

09 — Roadmap & Recommendations

Planned Features

Social Games

The codebase references social games as a future phase distinct from board games. Current infrastructure supports 1-on-1 sessions only.

What needs to change:

- Multi-user session model (room/lobby concept)
- A new consumer (SocialGameConsumer) or extended MatchmakingConsumer
- New Redis pool keys: matchmaking:pool:social_game:<code>
- New vibe value: "social_game"

Leaderboards / Profiles

GameResult rows are already stored. The data is ready. What's missing:

- A /leaderboard/ view
- A /profile/<username>/ view
- Aggregation queries (Sum of points_earned grouped by user)

More Game Codes

The GAME_LABELS dict has a commented "chess" entry. The plug-in system is ready — only need a JS engine and a DB row.

Technical Debt

#	Issue	Fix
1	Hardcoded Google OAuth Client ID	Audit settings.py — confirm GOOGLE_CLIENT_ID comes from environment only.
2	TURN server missing	Deploy coturn and add to iceServers in match_client.js. Consider fetching TURN credentials dynamically from a Django view for security.
3	No game state persistence	Store board state in Redis (keyed by session_id) and restore on reconnect via the sync message type.
4	Watchdog only on client side	Add a server-side TTL check or background task to sweep stale sessions older than N minutes.
5	SPOP race condition at scale	Use a Lua script to atomically pop + validate, or use a sorted set with timestamps.

#	Issue	Fix
6	CustomUser.password field conflict	Remove the explicit password field — Django's AbstractUser already handles it (128-char hashed field).
7	No rate limiting on WebSocket join	Track join attempts per connection; ignore subsequent joins if already in-session or waiting.
8	UserActivity table growth	Add periodic cleanup: delete activity logs older than 90 days.

Testing Gaps

Recommended test coverage:

- Consumer unit tests using channels.testing.WebsocketCommunicator
- Matchmaking integration test: two communicators joining the same pool → assert both receive matched
- Game result tests: verify GameResult.record_pair() creates correct rows for win/draw/forfeit
- Presence tests: assert Redis counters after connect/disconnect
- Self-match guard test: single user in pool should not match with themselves

Feature Wishlist

Feature	Complexity	Notes
Text filters / profanity guard	Low	Filter chat messages server-side
Geo-based matching	Medium	Add country to pool key
Gender/preference filters	Medium	Common in Omegle-style apps; adds moderation complexity
Virtual gifts / reactions	Low	Use custom message type
In-app reporting with screenshots	High	Requires frame capture + storage
Streaks / achievements	Medium	Build on GameResult
Mobile app	High	PWA or React Native
WebSocket reconnect / session resume	Medium	Redis session state + sync message
Admin moderation queue	Low	reported=True flag exists — just need a view